

```

1 %% Canonical PSO vs FIPS
2 % This code is based on the considerations made throughout the Section 5 of
3 % the related Thesis.
4
5 % This script compares three swarm configurations on three 2D benchmark
6 % functions (Sphere, Ackley, Rastrigin):
7 % 1) Canonical PSO      global topology, constriction factor
8 % 2) FIPS-Ring          unweighted FIPS, ring neighborhood topology
9 % 3) FIPS-All           unweighted FIPS, fully connected topology
10 %
11 % For each benchmark function, RUNS = 20 independent runs are executed.
12 % Within a given run, all three configurations start from the same
13 % random initial positions/velocities (X0, V0), so that any performance
14 % difference can be attributed to the algorithm/topology rather than
15 % to the initial conditions. The random seed is set once per run (based
16 % on the function index and run number, not on the algorithm), while the
17 % random sequences used afterwards for r1, r2 and rK during the velocity
18 % and position updates evolve indepependently for each configuration.
19 %
20 % For every run, the script records:
21 %     - the best fitness found at the end of the run (finalBest)
22 %     - whether/when the run satisfies the position-base success criterion,
23 %       under both the standard threshold and a stricter one (successRun/
24 %       itersToHit and successRunTight/itersToHitThigh)
25 %     - the mean convergence curve of the best fitness over iterations
26 %
27 % Results are summarized in a table and in per-function and combined plots
28 % on a logarithmic fitness scale.
29
30 % Author: Giorgia Colucci – Bachelor's Thesis, Politecnico di Torino
31
32 clear; clc; close all;
33 %% ----- GLOBAL PARAMETERS -----
34 D      = 2;           % search space dimensionality
35 N      = 30;          % swarm size
36 RUNS   = 20;          % independent runs per configuration
37 maxIter = 500;        % maximum number of iterations
38
39 % Canonical PSO parameters (constriction factor formulation, Eq. 3.4)
40 phi1 = 2.05;
41 phi2 = 2.05;
42 chi  = 0.7298;        % kappa = 1;
43
44 % FIPS parameters (unweighted, Eqs. 4.1, 4.3, 4.4)
45 phiFIPS = 4.1;
46 % same chi as PSO
47
48 % Success criterion
49 POS_TOL_FACTOR = 0.0015;
50 %% ----- FUNCTIONS SETUP -----
51 % Standard 2D domains. All three functions have their known global optimum
52 % at the origin, f(0,0) = 0.
53 funcNames = {'Sphere', 'Ackley', 'Rastrigin'};
54 lbAll     = [-100,      -32,      -5.12,];
55 ubAll     = [ 100,       32,       5.12,];
56
57 nFunc = length(funcNames);
58
59 % The three configurations compared for every function
60 confNames = {'Canonical PSO (global)', 'FIPS-Ring', 'FIPS-All'};
61 nConf     = length(confNames);

```

```

62
63 % Storage for the results (indices: function, configuration, run)
64 % 20-dimensional arrays, in which every element is a (nFunc x nConf)-dimensional matrix
65 finalBest = zeros(nFunc, nConf, RUNS);
66 successRun = false(nFunc, nConf, RUNS);
67 itersToHit = nan(nFunc, nConf, RUNS); % NaN is used when a run is unsuccessful
68 successRunTight = false(nFunc, nConf, RUNS);
69 itersToHitTight = nan(nFunc, nConf, RUNS);
70
71 % Storage for the mean convergence curves (function, configuration, iter)
72 meanCurve = zeros(nFunc, nConf, maxIter);
73
74 %% ----- MAIN SIMULATIONS LOOP -----
75 for fIdx = 1:nFunc
76     funcActual = funcNames{fIdx};
77     lb = lbAll(fIdx);
78     ub = ubAll(fIdx);
79     posTol = POS_TOL_FACTOR * (ub - lb);
80     posTolTight = 1e-4 * (ub - lb);
81
82     curveSum = zeros(nConf, maxIter); % (3 x maxIter) - dimensional matrix
83
84     for r = 1:RUNS
85
86         % --- Reproducible seed, different for every run ---
87         seed = 100*fIdx + r; % seed is related to the benchmark function and to the run
88         rng(seed);
89
90         % Random initial conditions, shared by ALL THREE configurations
91
92         X0 = lb + (ub - lb) * rand(N, D); % Initialize random positions within
93         V0 = (rand(N,D) - 0.5) .* (ub - lb); % Centering the random numbers to be
94         % from the interval [-0.5, 0.5] and scaling the random velocity in relation
95         % to the search space
96
97         % Configuration 1: Canonical PSO (global)
98         % For each run, bc is a (1*maxIter) vector containing the best
99         % fitness found at until every iteration. Then, the values of every run
100        % are added.
101        [bc, bd] = runPSO(funcActual, lb, ub, N, D, maxIter, chi, phi1, phi2, X0, V0);
102        finalBest(fIdx, 1, r) = bc(end); % contains, for each run, the value of
103        % until the end of the simulation
104        hitIdx = find(bd < posTol, 1, "first"); % find the index of the first elt
105        % that satisfies condition
106        if ~isempty(hitIdx) % following lines will be skipped if hitIdx is empty
107            successRun(fIdx, 1, r) = true; % true if criteria is reached
108            itersToHit(fIdx, 1, r) = hitIdx;
109        end
110        curveSum(1, :) = curveSum(1,:) + bc;
111
112        hitIdxTight = find(bd < posTolTight, 1, "first");
113        if ~isempty(hitIdxTight)
114            successRunTight(fIdx, 1, r) = true;
115            itersToHitTight(fIdx, 1, r) = hitIdxTight;
116        end
117
118        % Configuration 2: FIPS - Ring (same X0, V0)

```

```

118     [bc, bd] = runFIPS(funcActual, lb, ub, N, D, maxIter, 'Ring', chi, phiFIPS, X0,
V0);
119     finalBest(fIdx, 2, r) = bc(end);
120     hitIdx = find(bd < posTol, 1, "first");    % find the index of the first elt
that satisfies condition
121     if ~isempty(hitIdx)    % following lines will be skipped if hitIdx is empty
122         successRun(fIdx, 2, r) = true;    % true if criteria is reached
123         itersToHit(fIdx, 2, r) = hitIdx;
124     end
125     curveSum(2, :) = curveSum(2,:) + bc;
126
127     hitIdxTight = find(bd < posTolTight, 1, "first");
128     if ~isempty(hitIdxTight)
129         successRunTight(fIdx, 2, r) = true;
130         itersToHitTight(fIdx, 2, r) = hitIdxTight;
131     end
132
133     % Configuration 3: FIPS - All (same X0, V0)
134     [bc, bd] = runFIPS(funcActual, lb, ub, N, D, maxIter, 'All', chi, phiFIPS, X0,
V0);
135     finalBest(fIdx, 3, r) = bc(end);
136     hitIdx = find(bd < posTol, 1, "first");    % find the index of the first elt
that satisfies condition
137     if ~isempty(hitIdx)    % following lines will be skipped if hitIdx is empty
138         successRun(fIdx, 3, r) = true;    % true if criteria is reached
139         itersToHit(fIdx, 3, r) = hitIdx;
140     end
141     curveSum(3, :) = curveSum(3,:) + bc;
142
143     hitIdxTight = find(bd < posTolTight, 1, "first");
144     if ~isempty(hitIdxTight)
145         successRunTight(fIdx, 3, r) = true;
146         itersToHitTight(fIdx, 3, r) = hitIdxTight;
147     end
148 end
149
150     meanCurve(fIdx, :, :) = curveSum / RUNS;    % mean convergence curve for each
configuration:
151     % average of the minimum fitnesses found by the three configurations across all
the runs
152
153 end
154 %% ----- PRINT SUMMARY TABLE -----
155 fprintf('%-10s | %-22s | %-10s | %-11s | %-9s | %-7s | %-9s | %-8s | %-9s\n ', ...
156     'Function', 'Configuration', 'Mean best', 'Median best', 'Std best', ...
157     'Success', 'Med.iters', 'Succ-T', 'Med.iters-T');
158 fprintf(repmat('-', 1, 120)); fprintf('\n');
159
160 for fIdx = 1:nFunc
161     for cIdx = 1:nConf
162         % selects only the 20 run and eliminates the fIdx and cIdx
163         % dimension with squeeze, for calculating the mean, median and std
164         vals = squeeze(finalBest(fIdx, cIdx, :));
165         succ = squeeze(successRun(fIdx, cIdx, :));
166         iters = squeeze(itersToHit(fIdx, cIdx, :));
167         succTight = squeeze(successRunTight(fIdx, cIdx, :));
168         itersTight = squeeze(itersToHitTight(fIdx, cIdx, :));
169
170         medIters = median(iters(~isnan(iters)));    % eliminates all the nan values
171         if isnan(medIters)

```

```

172         medItersStr = 'never';           % if medIters is an empty arrays (every run had
nan iters)
173     else
174         medItersStr = sprintf('%.1f', medIters);
175     end
176
177     medItersTight = median(itersTight(~isnan(itersTight))); % eliminates all the
nan values
178     if isnan(medItersTight)
179         medItersStrTight = 'never';       % if medIters is an empty arrays (every
run had nan iters)
180     else
181         medItersStrTight = sprintf('%.1f', medItersTight);
182     end
183
184     fprintf('%-10s | %-22s | %-10.4g | %-11.4g | %-9.4g | %-7.2f | %-9s | %-8.2f |
%-9s\n', ...
185         funcNames{fIdx}, confNames{cIdx}, mean(vals), median(vals), std(vals), ...
186         mean(succ)*100, medItersStr, mean(succTight)*100, medItersStrTight);
187     end
188 end
189
190 %% ----- PLOTS -----
191 for fIdx = 1:nFunc
192
193     figure('Name', ['convergence_' funcNames{fIdx}]);
194     hold on;
195
196     plot(1:maxIter, squeeze(meanCurve(fIdx, 1, :)), 'k-', 'LineWidth', 2);
197     plot(1:maxIter, squeeze(meanCurve(fIdx, 2, :)), 'b-', 'LineWidth', 1.5);
198     plot(1:maxIter, squeeze(meanCurve(fIdx, 3, :)), 'r-', 'LineWidth', 1.5);
199
200     % for dark plot
201     % plot(1:maxIter, squeeze(meanCurve(fIdx, 1, :)), 'Color', '#00D7FF', 'LineWidth',
2, 'LineStyle', '--');
202     % plot(1:maxIter, squeeze(meanCurve(fIdx, 2, :)), 'Color', '#FFAA00', 'LineWidth',
2, 'LineStyle', '--');
203     % plot(1:maxIter, squeeze(meanCurve(fIdx, 3, :)), 'Color', '#FF55FF', 'LineWidth',
1.5);
204
205     set(gca, 'Yscale', 'log');           % uses a logarithmic scale for the y-axes
206     xlabel('Iteration');
207     ylabel('Mean best fitness (log scale)');
208     title(['Convergence on ' funcNames{fIdx} ' (mean over ' num2str(RUNS) ' runs)']);
209     legend(confNames);
210     grid on;
211     hold off;
212 end
213
214 %% ----- COMBINED FIGURE -----
215 figCombined = figure('Name', 'convergence_all', 'Position', ...
216     [100, 100, 1300, 1000]);
217
218 subplotIdx = {1, 2, [3, 4]};
219
220 if ~exist('figures', 'dir')
221     mkdir('figures');
222 end
223
224 for fIdx = 1:nFunc
225     subplot(2, 2, subplotIdx{fIdx});

```

```

226     hold on;
227
228     plot(1:maxIter, squeeze(meanCurve(fIdx, 1, :)), 'k-', 'LineWidth', 2);
229     plot(1:maxIter, squeeze(meanCurve(fIdx, 2, :)), 'b-', 'LineWidth', 1.5);
230     plot(1:maxIter, squeeze(meanCurve(fIdx, 3, :)), 'r-', 'LineWidth', 1.5);
231
232     % for dark plot
233     % plot(1:maxIter, squeeze(meanCurve(fIdx, 1, :)), 'Color', '#00D7FF', 'LineWidth',
2, 'LineStyle', '-');
234     % plot(1:maxIter, squeeze(meanCurve(fIdx, 2, :)), 'Color', '#FFAA00', 'LineWidth',
2, 'LineStyle', '--');
235     % plot(1:maxIter, squeeze(meanCurve(fIdx, 3, :)), 'Color', '#FF55FF', 'LineWidth',
2);
236
237     set(gca, 'Yscale', 'log');          % uses a logarithmic scale for the y-axes
238     xlabel('Iteration');
239     ylabel('Mean best fitness (log scale)');
240     title(['(' char(96 + fIdx) ') ' funcNames{fIdx}]);
241     legend(confNames);
242     grid on;
243     set(gca, 'FontSize', 11);
244     hold off;
245
246     lgd = legend(confNames, 'Orientation', 'horizontal');
247     lgd.Position(1) = 0.5 - lgd.Position(3)/2;      % horizontal centering
248     lgd.Position(2) = 0.02;                        % just above the bottom edge
249 end
250
251 exportgraphics(figCombined, fullfile('figures', 'convergence_all.png'), 'Resolution',
300);
252
253 %% ----- PRESENTATION FIGURE -----
254 figCombined = figure('Name', 'convergence_all_presentation', ...
255     'Position', [100, 100, 1600, 500]);
256
257 t = tiledlayout(1, 3, ...
258     'TileSpacing', 'normal', ...
259     'Padding', 'compact');
260
261 for fIdx = 1:nFunc
262
263     ax = nexttile;
264     hold on;
265
266     % Canonical PS0
267     p1 = plot(1:maxIter, squeeze(meanCurve(fIdx, 1, :)), ...
268         'k-', 'LineWidth', 2);
269
270     % FIPS-Ring
271     p2 = plot(1:maxIter, squeeze(meanCurve(fIdx, 2, :)), ...
272         'b-', 'LineWidth', 1.5);
273
274     % FIPS-All
275     p3 = plot(1:maxIter, squeeze(meanCurve(fIdx, 3, :)), ...
276         'r-', 'LineWidth', 1.5);
277
278     set(gca, 'YScale', 'log');
279
280     xlabel('Iteration');
281     ylabel('Mean best fitness');
282

```

```

283     title(funcNames{fIdx});
284
285     grid on;
286     box on;
287
288     set(gca, 'FontSize', 12);
289
290     % Save handles only from the first subplot
291     if fIdx == 1
292         legendHandles = [p1 p2 p3];
293     end
294
295     hold off;
296 end
297
298 % One common legend for the entire figure
299 lgd = legend(legendHandles, confNames, ...
300     'Orientation', 'horizontal');
301
302 lgd.Layout.Tile = 'south';
303
304 % Export
305 if ~exist('figures', 'dir')
306     mkdir('figures');
307 end
308
309 exportgraphics(figCombined, ...
310     fullfile('figures', 'convergence_all_presentation.png'), ...
311     'Resolution', 300);
312 %% =====
313 % BENCHMARK FUNCITONS (2D formulations, Eqs. 5.1–5.3 of the thesis)
314 % =====
315 % Input:     function name funcActual and a N-by-2 matrix X (one particle
316 % per row).
317 % Output:    N-by-1 vector representing the evaluations for every particle.
318 % X is always within [lb, ub] with the reflecting-wall strategy adopted.
319
320 function fval = evaluateFunc(funcActual, X)
321     x1 = X(:, 1);
322     x2 = X(:, 2);
323
324     % when funcActual corresponds to the case_expression, the switch function
325     % performs the corresponding statements and exits the switch lock.
326     switch funcActual
327     case 'Sphere'
328         fval = x1.^2 + x2.^2; % Sphere
329     case 'Ackley'
330         a1 = sqrt(0.5 * (x1.^2 + x2.^2));
331         a2 = 0.5 * (cos(2*pi*x1) + cos(2*pi*x2));
332         fval = 20 + exp(1) - 20*exp(-0.2*a1) - exp(a2); % Ackley
333     case 'Rastrigin'
334         fval = 20 + x1.^2 + x2.^2 - 10*cos(2*pi*x1) - 10*cos(2*pi*x2); % Rastrigin
335     function evaluation
336     otherwise
337         error('Unknown benchmark function: %s', funcActual)
338     end
339 end
340 %% =====

```

```

341 % NEIGHBORHOOD TOPOLOGY (used by FIPS only)
342 % =====
343 % Returns the particle indices belonging to the neighborhood of particle i,
344 % including particle i itself. For Ring topology, neighborhood of particle
345 % i has 3 members {i-1, i, i+1}. For All topology, has N members.
346
347 function idxNeigh = getNeigh(topology, i , N)
348     if strcmp(topology, 'Ring')
349         left = i - 1;
350         if left < 1; left = N; end
351         right = i + 1;
352         if right > N; right = 1; end
353         idxNeigh = [left, i, right];
354     else % 'All'
355         idxNeigh = 1:N; % For 'All' topology, all particles are neighbors
356     end
357 end
358
359 %% =====
360 % CANONICAL PSO, global version (constriction factor formulation, Eq. 3.4)
361 % =====
362 function [bestCurve, bestDist] = runPSO(funcActual, lb, ub, N, D, ...
363     maxIter, chi, phi1, phi2, X, V)
364
365     Pbest = X; % initialize the BEST POSITION with X
366     PbestVal = evaluateFunc(funcActual, X); % FITNESS of the best position
367
368     bestCurve = zeros(1, maxIter); % (1 x maxIter) - dimensional vector
369     bestDist = zeros(1, maxIter); % distance from the known optimum
370
371     for t = 1:maxIter
372
373         % --- Global best used for velocity update ---
374         [~, gBestIdx] = min(PbestVal); % index of the best fitness value
375         Gbest = Pbest(gBestIdx, :); % position (x1, x2) of the global best
376
377         % --- Velocity and positions update (Eq. 3.4) ---
378         r1 = rand(N, D); % random values for every N particle and for the 2
dimensions
379         r2 = rand(N, D);
380         V = chi * (V + phi1 * r1 .* (Pbest - X) + phi2 * r2 .* (Gbest - X));
381         X = X + V;
382
383         % --- Reflecting wall boundary ---
384         % The velocity component that caused the boundary violation is
385         % reversed, and the position is clipped to the nearest bound
386         hitLow = X < lb; % assign true only to the component outside the bound
387         hitHigh = X > ub;
388         V(hitLow | hitHigh) = -V(hitLow | hitHigh); % change only component where the
condition is true
389
390         X = min(max(X, lb), ub); % clip the position outside the boundary
391
392         % --- Update personal bests AFTER the t velocity and position update ---
393         fitVal = evaluateFunc(funcActual, X);
394         for i = 1:N
395             if fitVal(i) < PbestVal(i)
396                 Pbest(i, :) = X(i, :); % Update best position of the single particle
397                 PbestVal(i) = fitVal(i); % Update best fitness of the single particle
398             end
399         end

```

```

400
401     % --- Global best position found so far AFTER iteration t ---
402     % PbestVal contains time memory; while, bestVal is done among all particles
403     [bestVal, gBestIdx] = min(PbestVal);           % minimum fitness and index of the
best fitness
404                                                     % value among that of all the
particles
405     Gbest = Pbest(gBestIdx, :);                   % position (x1, x2) of the global
best
406
407     % --- Best curve and distance ---
408     % Register, at every iteration, the quality of the solution found until now
409     bestCurve(t) = bestVal;                       % minimum value of the best fitness value
until iteration t
410     bestDist(t) = max(abs(Gbest));                % calculates max(|x_best|, |y_best|), i.e
the distance
411                                                     % in INFINITY NORM. otherwise, can be used
norm(Gbest)
412     end
413 end
414
415 %% =====
416 % FULLY INFORMED PARTICLE SWARM, unweighted (Eqs. 4.3 and 5.4)
417 % =====
418 function [bestCurve, bestDist] = runFIPS(funcActual, lb, ub, N, D, ...
419     maxIter, topology, chi, phi, X, V)
420
421     Pbest      = X;                               % initialize the BEST POSITION with X
422     PbestVal    = evaluateFunc(funcActual, X);      % FITNESS of the best position
423
424     bestCurve = zeros(1, maxIter);
425     bestDist  = zeros(1, maxIter);                % distance from the known optimum
426
427     for t = 1:maxIter
428         Vnew = zeros(N, D);
429
430         % --- Velocity and positions update (Eqs. 4.3 and 5.4) ---
431         for i = 1:N
432             idxNeigh = getNeigh(topology, i, N);
433             K = length(idxNeigh);
434             contribution = zeros(1,D);
435
436             % Unweighted FIPS: average contribution of every neighbor
437             for j = 1:K
438                 rj = rand(1, D);                   % vector 1xD with random numbers from
uniform distribution [0,1]
439                 phi_j = rj * (phi/K);              % scaling with (phi/K)
440                 contribution = contribution + phi_j .* (Pbest(idxNeigh(j), :) - X(i, :));
441             end
442
443             Vnew(i, :) = chi * (V(i, :) + contribution);
444         end
445
446         V = Vnew;
447         X = X + V;
448
449         % --- Reflecting wall boundary ---
450         % The velocity component that caused the boundary violation is
451         % reversed, and the position is clipped to the nearest bound
452         hitLow = X < lb;                            % assign true only to the component outside the bound
453         hitHigh = X > ub;

```

```

454
455         V(hitLow | hitHigh) = -V(hitLow | hitHigh);    % change only component where the
condition is true
456         X = min(max(X, lb), ub);    % clip the position outside the boundary
457
458         % --- Update personal bests AFTER the t velocity and position update ---
459         fitVal = evaluateFunc(funcActual, X);
460         for i = 1:N
461             if fitVal(i) < PbestVal(i)
462                 Pbest(i, :) = X(i, :);    % Update best position of the single particle
463                 PbestVal(i) = fitVal(i);    % Update best fitness of the single particle
464             end
465         end
466
467         % --- Global best position found so far AFTER iteration t ---
468         [bestVal, bestIdx] = min(PbestVal);    % minimum value and index of the best
fitness
469                                         % value among that of all the
particles
470         bestPos = Pbest(bestIdx, :);    % position (x1, x2) of the global best
471
472         % --- Best curve and distance ---
473         % Register, at every iteration, the quality of the solution found until now
474         bestCurve(t) = bestVal;    % minimum value of the best fitness value
475         bestDist(t) = max(abs(bestPos));    % calculates max(|x_best|, |y_best|), i.e
the distance
476                                         % in infinity norm. otherwise, can be used
norm(Gbest)
477     end
478 end

```